

shogiAI 數據科學理論與應用報告

Policy-only 將棋模型、資料前處理與 Alpha-beta 搜尋

shogiAI 專案

2026 年 6 月

摘要

shogiAI 專案以 CSA 將棋棋譜與 MySQL 資料庫作為資料來源，將「局面、下一手」轉換成可監督學習資料。局面以 $43 \times 9 \times 9$ 張量表示，走法以 13,689 類 move label 表示，模型採用 policy-only 卷積神經網路預測實戰下一手並排序合法候選手。Value head 與 root value bonus 已移除，局面優劣由 alpha-beta 搜尋、quiescence search、transposition table、killer move、history heuristic 與手工評估函數處理。截至 2026-06-23，遠端 MySQL 資料庫包含 80,480 局、9,331,387 筆 positions 與 9,250,907 筆 moves；目前完成訓練的 policy-only 模型使用 2026-06-16 匯出的 4,855,173 筆 position-policy 樣本，測試 Top-1 accuracy 為 41.103%，Top-3 accuracy 為 65.239%，Top-5 accuracy 為 74.489%。

目录

1	數據科學目標	4
2	問題定義	4
3	資料來源與截至 2026-06-23 的規模	4
4	資料前處理流程	5
5	局面特徵編碼	6
6	行棋方視角旋轉	7
7	走法標籤設計	7
7.1	一般移動	7
7.2	打入持駒	7
7.3	總類別數	8

目錄	2
8 合法手規則與 Rule-based Mask	8
9 從 Policy/Value 改成 Policy-only	8
9.1 Value 標籤訊號太粗	9
9.2 投降資料使局面價值不精準	9
9.3 小型 CNN 容量有限	9
9.4 實際應用以候選手排序為主	9
10 Policy-only 模型架構	9
10.1 卷積層的原理	10
10.2 為什麼使用 CNN 而不是只用全連接層	10
10.3 激活函數與 ReLU 的理由	10
11 訓練方法	11
11.1 Loss Function	11
11.2 Optimizer	12
11.3 Learning Rate Scheduler	12
11.4 Game-level Split	12
12 訓練結果	12
13 結果解讀	13
14 模型推論與快取	13
15 Alpha-beta 搜尋概論	14
15.1 Minimax 與 Negamax	14
15.2 Alpha-beta Pruning	14
16 shogiAI 搜尋技術	15
16.1 Quiescence Search	15
16.2 Transposition Table	15
16.3 Killer Move	15
16.4 History Heuristic	15
16.5 Late Move Reduction	15
17 手工評估函數	15
18 Policy Network 如何結合 Alpha-beta	16
19 Opening Book	17

目录	3
20 模型互打與評估	17
21 前端展示與應用	18
22 限制與改進方向	18
22.1 模型容量限制	18
22.2 資料標籤限制	18
22.3 Legal Move Mask	18
22.4 自我對弈	18
22.5 Value 的未來可能	18
23 結論	19
A 常用指令	19
A.1 匯出資料集	19
A.2 訓練 policy-only 模型	19
A.3 啟動 AI 瀏覽器	19
A.4 模型互打	20

1 數據科學目標

shogiAI 的數據科學目標是讓模型從人類實戰棋譜中學會「某個局面下，下一手最可能怎麼走」。目前模型採用 policy-only 設計，也就是模型只輸出走法機率，不輸出局面勝率或 value。

模型可表示為：

$$f_{\theta}(s) = p$$

其中 s 是目前局面， p 是長度 13,689 的 policy logits。每一個 logit 對應一種可能走法。推論時，系統只取目前局面的合法手，讀取這些合法手對應的 policy 分數，再排序給 alpha-beta 搜尋使用。

2 問題定義

本專案的機器學習任務是多類別分類。

項目	內容
輸入	將棋局面，編碼為 $43 \times 9 \times 9$ 張量。
輸出	下一手 move label，總類別數為 13,689。
標籤來源	人類實戰棋譜中的下一手。
學習方法	Supervised learning / imitation learning。
Loss	CrossEntropyLoss。
應用方式	排序合法手，輔助 alpha-beta 搜尋。

這個任務不等於直接訓練一個能獨立下棋的模型。將棋局面往往有許多合理手，實戰下一手也不一定是唯一最佳手。因此模型的主要價值是把合理候選手排到前面，讓搜尋更有效率。

3 資料來源與截至 2026-06-23 的規模

資料來源包含本地 CSA 檔案、遠端爬蟲蒐集的 CSA 檔案，以及 MySQL 中已匯入的棋譜資料。遠端資料庫持續由爬蟲與 watcher 匯入資料；目前完成訓練的 policy-only 模型使用 2026-06-16 由 MySQL 匯出的資料集。

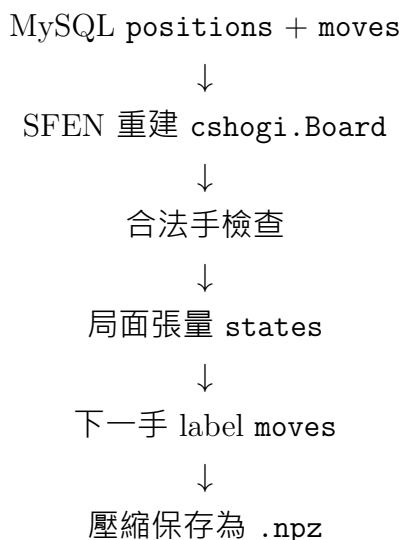
項目	數值
匯出資料集	out/policy_dataset.policy_only_latest.npz
對局數	42,534 局

項目	數值
訓練樣本總數	4,855,173 筆
非法資料列	0 筆
TORYO 樣本	4,829,692 筆
DRAW 樣本	104 筆
SENNICHITE 樣本	25,377 筆
資料集大小	約 332 MB
訓練資料匯出時間	2026-06-16 22:24:07 +0800
模型訓練完成時間	2026-06-16 22:57:57 +0800

這些樣本是從資料庫的 `positions` 與 `moves` join 得到。每筆樣本代表「某一手前的局面」與「該局面下實戰走出的下一手」。截至 2026-06-23 21:45，遠端資料庫本身已累積 80,480 局、9,331,387 筆 `positions`、9,250,907 筆 `moves`、5,143 位 `players` 與 2 筆 `users`；資料庫最晚匯入時間為 2026-06-23 04:56:24。由於模型訓練使用的是 2026-06-16 匯出的固定資料集，訓練結果與目前資料庫總量分開記錄。

4 資料前處理流程

資料前處理主要由 `src/export_policy_dataset.py` 與 `src/csa_preprocess.py` 負責。整體流程如下：



核心 SQL 邏輯是將局面與下一手對齊：

```

SELECT p.game_id, p.move_number, p.sfen, m.usi_move, g.result
FROM positions p
JOIN moves m
  ON m.game_id = p.game_id

```

```

AND m.move_number = p.move_number + 1
JOIN game_records g
  ON g.game_id = p.game_id
ORDER BY p.game_id, p.move_number;

```

在 Python 中，系統會用 `cshogi.Board(sfen)` 還原棋盤，用 `board.move_from_usi(usi_move)` 取得 `move`，再以 `board.is_legal(move)` 檢查合法性。非法資料不會進入訓練集。

5 局面特徵編碼

神經網路不能直接讀 SFEN 字串，因此系統將局面轉成固定大小張量：

$$state \in \mathbb{R}^{43 \times 9 \times 9}$$

43 個 planes 的意義如下：

Plane 範圍	意義
0–13	自方 14 種棋子在棋盤上的位置。
14–27	對方 14 種棋子在棋盤上的位置。
28–34	自方 7 種持駒數量。
35–41	對方 7 種持駒數量。
42	行棋方 plane。

這種表示法類似影像中的多通道資料。棋盤是 9×9 空間，棋種、陣營、持駒與行棋方則作為 channel。CNN 可以在這種表示中學習局部形狀，例如攻防、升變區、玉周圍防守與棋子協同。

這裡選擇 plane-based encoding，而不是直接使用 SFEN 字串或單一整數矩陣，主要有三個理由。第一，神經網路對固定維度數值張量最容易處理；SFEN 雖然人類可讀，但不同局面字串長度不同，且字元之間沒有明確的空間鄰近關係。第二，不同棋種若只用不同整數表示，模型會誤以為棋種之間存在連續大小關係，例如「飛車的數字比步兵大」這種編碼上的大小不一定等於語意上的距離。使用 one-hot planes 可以讓每種棋子都有獨立通道，避免把類別資料錯當成連續數值。第三，將棋的棋盤關係具有空間性，攻擊、防守、釘住、升變區與玉周圍形狀都和相鄰格子有關，plane-based encoding 能讓卷積層直接利用這種局部結構。

持駒也被放入 plane 中，是因為將棋和西洋棋不同，吃到的棋子可以重新打入棋盤。持駒不是棋盤上的位置，但它會改變所有空格的潛在威脅。例如手上有步、金或銀時，對敵玉周圍的打入壓力完全不同。因此系統用固定 planes 表示雙方持駒數量，讓模型在看棋盤形狀時，也能同時知道「手上有哪些可用資源」。

6 行棋方視角旋轉

專案預設 `orient_to_turn=True`。若輪到後手行棋，系統會把棋盤旋轉 180 度，讓模型永遠從「目前要下棋的一方」的視角看盤。

這個設計有三個好處：

1. 減少先手與後手對稱造成的學習負擔。
2. 讓同一戰型在不同顏色下能共享特徵。
3. 讓模型輸出永遠表示「自己下一手」而不是固定先手或後手。

例如後手局面會被轉成自方在下方的視角，後手的棋子會進入「自方棋子」planes。這使 policy network 不必另外學一套後手座標系。

視角旋轉也可以視為一種資料標準化。若不旋轉，模型必須分別學會「先手從下往上進攻」與「後手從上往下進攻」兩套近似對稱的模式；旋轉後，模型只需要學「自方向前進攻」這一套模式。這會降低資料需求，也讓同樣的戰型、圍玉或攻擊形狀更容易被共享。

7 走法標籤設計

Policy head 需要固定輸出維度，但每個局面的合法手數量不同。因此系統將所有可能走法編成固定 label，再在推論時只篩選合法手。

7.1 一般移動

一般移動由來源格、目標格與是否升變組成：

$$81 \times 81 \times 2 = 13,122$$

其中 81 是棋盤格數，乘以 2 是因為同一個來源與目標可能有升變與不升變兩種。

7.2 打入持駒

將棋有持駒打入規則。可打入棋子共有 7 種，目標格有 81 格：

$$7 \times 81 = 567$$

7.3 總類別數

$$13,122 + 567 = 13,689$$

因此模型最後一層輸出 13,689 維 logits。訓練時使用實戰下一手的 label 作為 target。

這種 label 設計的優點是簡單、固定且容易和 cross entropy 搭配。神經網路每次都輸出同樣長度的向量，不需要因不同局面的合法手數量不同而改變輸出形狀。缺點是其中很多類別在某個局面下其實不合法，因此推論階段必須搭配規則系統，只保留目前 `cshogi.Board` 判定為合法的走法。換句話說，模型負責學習「哪一類走法像人類高手會下」，規則引擎負責保證「這一手在將棋規則下真的能走」。

8 合法手規則與 Rule-based Mask

將棋有許多嚴格規則，例如棋子走法、升變限制、持駒打入、二步禁手、打步詰禁手與王手應對。若完全交給神經網路自己學會所有規則，模型需要大量資料才能避免非法手，而且仍可能在罕見局面犯錯。因此 shogiAI 採用 rule-based legality check，也就是由 `cshogi` 根據將棋規則列出合法手，再把 policy network 的分數套用到這些合法手上。

這樣分工有明確理由：

- **規則是確定性的**：合法手判斷不需要從資料中猜測，直接使用規則引擎可以得到正確結果。
- **模型容量應留給棋感**：小型 CNN 的能力有限，應優先學習候選排序、戰型偏好與局部攻防，而不是浪費容量記憶基本規則。
- **避免非法輸出**：即使模型對非法 label 給高分，合法手 mask 也會將其排除，實際 AI 不會走出非法手。
- **提升機率解釋性**：對合法手集合做 softmax 後，機率代表「在目前能走的手裡，模型偏好哪一手」。

因此，rule-based mask 不是神經網路之外的補丁，而是棋類 AI 中常見的安全邊界設計。規則處理「能不能走」，神經網路處理「像不像好手」，alpha-beta 搜尋再處理「往後算是否真的有效」。

9 從 Policy/Value 改成 Policy-only

專案早期曾設計 policy/value network，模型同時輸出下一手 policy 與局面 value。Value head 的概念是用最終勝負作為標籤，讓模型學習目前行棋方最後會贏、輸或和棋：

$$z \in \{-1, 0, 1\}$$

實作與測試後，專案改成 policy-only。主要原因如下。

9.1 Value 標籤訊號太粗

Value label 來自一盤棋最後結果，但中盤局面距離終局很遠。某一方最後獲勝，不代表他在中盤每一手都真的佔優。用最終勝負硬套到所有中間局面，會讓 value target 含有大量噪音。

9.2 投降資料使局面價值不精準

許多棋譜以 TORYO 結束。投降能指出勝負方，但無法提供每個中間局面的精確勝率。尤其在將棋中，局面可能因一手戰術而突然逆轉，用整局結果回填所有局面會讓 value head 學到不穩定訊號。

9.3 小型 CNN 容量有限

目前模型是輕量 CNN，適合快速訓練與排序候選手，但要準確評估將棋複雜局面需要更深模型、更多搜尋標籤或自我對弈。若同時要求模型學 policy 與 value，可能分散有限模型容量。

9.4 實際應用以候選手排序為主

本專案 AI 最終仍使用 alpha-beta 搜尋計算變化。神經網路最需要做的是把好手排到前面，提高剪枝效率。局面評估則由傳統評估函數、搜尋深度與 quiescence search 負責。因此移除 value head 後，系統更簡單，也更符合目前需求。

10 Policy-only 模型架構

模型定義於 src/policy_model.py，名稱為 SmallPolicyNet。架構如下：

```
Conv2d(43, 64, kernel_size=3, padding=1)
ReLU
Conv2d(64, 64, kernel_size=3, padding=1)
ReLU
Conv2d(64, 32, kernel_size=3, padding=1)
ReLU
Flatten
Linear(32 * 9 * 9, 512)
```

ReLU

Linear(512, 13689)

前半段卷積層保留棋盤空間結構，後半段 fully connected layer 將特徵轉成所有 move labels 的 logits。目前模型沒有 value head，因此只輸出 policy。

10.1 卷積層的原理

卷積神經網路適合棋盤資料，是因為棋盤具有明確的二維空間結構。卷積層會使用小型 filter，也就是 kernel，在棋盤上滑動。每個 kernel 只看局部區域，例如 3×3 範圍，並對輸入 channels 做加權加總：

$$y_{k,i,j} = b_k + \sum_c \sum_{u=-1}^1 \sum_{v=-1}^1 w_{k,c,u,v} x_{c,i+u,j+v}$$

其中 x 是輸入 plane， w 是卷積核權重， b_k 是偏置， $y_{k,i,j}$ 是第 k 個輸出特徵在位置 (i, j) 的值。直觀來說，一個 kernel 可能學到「玉周圍是否有金銀保護」、另一個 kernel 可能學到「某條線上是否有飛車壓力」、也可能學到「桂馬跳躍攻擊形狀」。

本專案使用 kernel_size=3 與 padding=1。 3×3 kernel 足以捕捉相鄰格子的局部關係，padding 則讓輸出仍維持 9×9 ，避免棋盤邊界在每層卷積後縮小。多層 3×3 卷積堆疊後，模型的感受野會逐層擴大：第一層看到鄰近格，第二層可間接看到更大範圍，第三層則能組合出更複雜的攻防圖樣。

10.2 為什麼使用 CNN 而不是只用全連接層

如果直接把 $43 \times 9 \times 9$ 展平成向量再接 fully connected layer，模型可以學習任何位置組合，但會失去「相鄰格子有關係」這個先驗，也會需要更多參數。CNN 的優點是參數共享：同一個 kernel 會在整個棋盤上重複使用，因此模型能學到「某種局部形狀」並在不同位置辨識它。這對將棋很重要，因為同樣的銀冠形狀、金銀連結、飛車橫利或角道壓力，可能出現在棋盤不同位置。

因此，本專案前半段用卷積層抽取空間特徵，後半段才 flatten 並接 fully connected layer，把整個棋盤的資訊轉換成 13,689 個走法類別的分數。這是「局部特徵抽取後，再進行全局決策」的設計。

10.3 激活函數與 ReLU 的理由

卷積和全連接本身都是線性運算。如果多層網路中間沒有激活函數，整個模型即使堆很多層，數學上仍可合併成一個線性轉換，無法表示複雜的非線性棋局判斷。激活函數的作用就是在每層之間加入非線性，使模型可以學到更複雜的規則與模式。

本模型使用 ReLU：

$$\text{ReLU}(x) = \max(0, x)$$

ReLU 的設計很簡單：正值保留，負值歸零。它的優點包括：

- **計算快**：只需要比較是否大於 0，適合大量訓練與快速推論。
- **減少梯度消失**：當輸入為正時，梯度為 1，比 sigmoid 或 tanh 在飽和區更容易訓練深層網路。
- **產生稀疏特徵**：負值被歸零，代表某些特徵在某些局面中不啟用，這有助於模型只關注目前有用的攻防形狀。
- **適合輕量模型**：ReLU 沒有額外參數，和小型 CNN 搭配時簡單穩定。

使用 ReLU 並不是因為它能直接表達將棋規則，而是因為它讓卷積層抽出的局部訊號可以被非線性組合。例如「某棋子被攻擊」和「該棋子有保護」同時出現時，局面意義不是兩個分數簡單相加；非線性激活能讓後續層學到這種條件式互動。

11 訓練方法

訓練腳本為 `src/train_policy.py`。目前訓練流程只包含 policy 分類。

11.1 Loss Function

使用 cross entropy：

$$\mathcal{L}_{policy} = -\log \frac{\exp(\hat{p}_a)}{\sum_j \exp(\hat{p}_j)}$$

其中 a 是實戰下一手的 move label， $\hat{p}$ 是模型輸出的 logits。

Logits 不是機率，而是模型對每個走法類別的原始分數。Softmax 會將 logits 轉成機率分布：

$$P(a = j \mid s) = \frac{\exp(\hat{p}_j)}{\sum_k \exp(\hat{p}_k)}$$

Cross entropy 的意義是：若實戰下一手是 a ，模型就應該把 a 的機率拉高。當模型給實戰走法的機率越低，loss 越大；當模型越相信實戰走法，loss 越小。這很適合 policy learning，因為資料標籤本身就是「在此局面中，人類實戰選擇的下一手」。

不過，cross entropy 也有一個限制：它把資料中的下一手當成唯一正解，但將棋局面常常有多個合理手。因此 Top-1 accuracy 不能完全代表棋力。對本專案來說，Top-3、Top-5 更重要，因為 policy network 的任務是協助搜尋排序，只要把好手放在前面，alpha-beta 就有機會進一步驗證。

11.2 Optimizer

使用 AdamW。AdamW 將 adaptive gradient 與 weight decay 分離，通常比傳統 Adam 加 L2 regularization 更穩定。

AdamW 同時追蹤梯度的一階動量與二階動量。一階動量類似「最近梯度方向的平均」，能讓更新方向更穩；二階動量會估計每個參數梯度大小，讓不同參數有不同有效 learning rate。將棋 policy 分類有 13,689 個輸出類別，梯度分布可能很不均勻，AdamW 對這種 sparse、多類別問題通常比單純 SGD 更容易穩定收斂。

Weight decay 的作用是限制權重過大，降低過擬合。若模型只記住訓練棋譜中的局部特例，在測試局面或實際對弈中就可能泛化不好。AdamW 將 weight decay 和 Adam 的 adaptive update 分離，使正則化效果更符合直覺。

11.3 Learning Rate Scheduler

使用 CosineAnnealingLR。Learning rate 從初始值逐漸降低到最小值，使前期快速學習，後期微調模型。

Cosine annealing 的曲線在前期下降較平滑，後期逐漸接近最小 learning rate。前期模型參數尚未形成有效特徵，需要較大的 learning rate 快速移動；後期模型已接近較佳區域，較小 learning rate 可以避免在最小值附近震盪。這對 policy network 很重要，因為輸出類別多，若後期 learning rate 過大，Top-k accuracy 可能在驗證集上反覆波動。

11.4 Game-level Split

資料切分不是隨機切每一手，而是依 game_id 切分。原因是同一盤棋相鄰局面非常相似，如果同一盤棋的前半放進訓練集、後半放進測試集，測試結果會過度樂觀。Game-level split 可以降低資料洩漏。

12 訓練結果

policy-only 模型在遠端大數據主機訓練，並已同步回本地 models/policy_model.pt。截至 2026-06-23 查核，遠端與本地模型檔案仍對應同一組訓練結果。模型 SHA-256 為：

79bee4f2299667004803ec2265ba67a06c3236d08735372a7594a711b34b0ab0

項目	數值
Train samples	3,882,230
Validation samples	487,292
Test samples	485,651
Test policy loss	2.2751

項目	數值
Top-1 accuracy	41.103%
Top-3 accuracy	65.239%
Top-5 accuracy	74.489%

初始局面的前五個 policy 候選手如下：

走法	機率
7g7f	61.10%
2g2f	33.92%
5g5f	3.34%
2h7h	1.15%
9g9f	0.15%

13 結果解讀

Top-1 accuracy 表示模型直接猜中實戰下一手的比例。目前模型 Top-1 為 41.103%，代表在測試局面中，模型第一候選手有約四成與實戰相同。

Top-5 accuracy 更適合評估本專案用途。將棋每個局面常有多個合理手，而模型主要用於搜尋前的 move ordering。Top-5 為 74.489%，表示實戰下一手大約四分之三會出現在模型前五候選手中，這對 alpha-beta 剪枝很有幫助。

從應用角度看，Top-1 accuracy 回答的是「模型能不能直接模仿人類下一手」，Top-5 accuracy 回答的是「模型能不能把人類實戰手放進少數優先候選」。由於 shogiAI 並不是直接選 policy 最大的一手，而是把 policy 作為搜尋排序，Top-5 更貼近實際系統效果。只要合理手排得夠前面，搜尋就能更早得到好的 α 或 β 邊界，進而剪掉大量低價值分支。

14 模型推論與快取

推論封裝為 PolicyPredictor。它負責：

- 載入 .pt checkpoint。
- 將 cshogi.Board 轉成 $43 \times 9 \times 9$ state。
- 執行模型取得 logits。
- 只取合法手對應的 label 分數。

- 對合法手分數做 softmax，得到候選手機率。
- 依 score 排序回傳 PolicyCandidate。
- 用 `board.zobrist_hash()` 做 prediction cache，減少重複局面推論。

這裡的 softmax 只在合法手集合內計算，因此機率表示「在目前合法手中，模型偏好此手的比例」。

這裡再次使用合法手集合，是因為 13,689 維輸出中，大多數 label 在單一局面下不合法。若直接對全部 label 做 softmax，非法手也會分走機率質量，使合法手之間的相對分數較難解釋。先用規則列出合法手，再只對合法手 logits 做 softmax，可以得到更符合實際決策的機率。

15 Alpha-beta 搜尋概論

只靠 policy network 直接選最高機率手，容易漏看短期戰術，例如吃子交換、王手、詰將棋或防守手。因此 shogiAI 將神經網路與傳統搜尋結合。搜尋核心位於 `src/ai_search.py`。

15.1 Minimax 與 Negamax

棋類遊戲是雙人零和遊戲。若一方得分越高，另一方就越低。Minimax 的想法是：我方選擇能讓自己最好的走法，對方也會選擇讓我最差的應手。

Negamax 是 minimax 的簡化形式，利用零和性質：

$$score(s) = \max_a -score(T(s, a))$$

其中 $T(s, a)$ 表示在局面 s 下走 a 後的新局面。每往下一層，視角會切換到對手，所以分數取負。

15.2 Alpha-beta Pruning

Alpha-beta 是 minimax 的剪枝最佳化。它維護兩個界線：

- α ：目前已知我方至少能拿到的分數。
- β ：目前對方允許我方拿到的上限。

若搜尋中發現某分支已不可能影響最終選擇，就停止展開該分支。剪枝不改變理論結果，但可大幅減少節點數。

Alpha-beta 的效率和走法順序高度相關。若最佳手越早被搜尋， α 或 β 越快變嚴格，後續分支越容易被剪掉；若好手排在很後面，搜尋會先浪費大量時間檢查較差走法。這就是 policy network 能發揮價值的地方：它不需要直接判斷最終勝負，只要提供較好的 move ordering，就能讓 alpha-beta 在同樣時間內搜尋更深或搜尋更多關鍵變化。

16 shogiAI 搜尋技術

16.1 Quiescence Search

普通 alpha-beta 若剛好在戰術交換中停止，可能出現 horizon effect。例如葉節點看起來我方多一子，但下一手對方能吃回。Quiescence search 會在葉節點繼續延伸戰術手，例如吃子、升變或王手，直到局面較安靜再評估。

16.2 Transposition Table

不同走法順序可能到達同一局面。Transposition table 用 Zobrist hash 記錄已搜尋局面的分數、深度與最佳手。若搜尋再次遇到相同局面，可以直接使用快取結果或縮小 alpha-beta 範圍。

16.3 Killer Move

Killer move heuristic 記錄在同一搜尋深度曾造成 beta cutoff 的安靜手。若某手曾讓對手無法接受，之後同層搜尋可優先嘗試它，提高剪枝機會。

16.4 History Heuristic

History heuristic 統計某個走法在過去搜尋中造成 cutoff 的頻率與深度。越常在深層造成 cutoff 的走法，排序分數越高。

16.5 Late Move Reduction

若走法排序已經把較可能的好手放前面，後面排序很低且不是王手、不是戰術手的走法，可以先用較低深度搜尋。若低深度結果意外很好，再回頭完整搜尋。這可以節省大量時間。

17 手工評估函數

搜尋到葉節點時，需要評估局面分數。evaluate_position 不是只算子力，還加入多種將棋特徵：

- **子力價值**：飛、角、金、銀、桂、香、步與升變子的基礎價值。
- **持駒價值**：將棋持駒可打入，因此持駒也有分數。
- **位置分數**：棋子靠近中央、進入升變區或位置活躍會加分。
- **機動力**：可行走法越多，局面越靈活。

- **玉安全**：玉周圍保護、敵方壓力與直線攻擊。
- **懸空棋子**：被攻擊但保護不足的棋子會扣分。
- **持駒壓力**：持駒對敵玉周圍可打入威脅會加分。
- **節奏**：輪到行棋方可能有主動權 bonus。

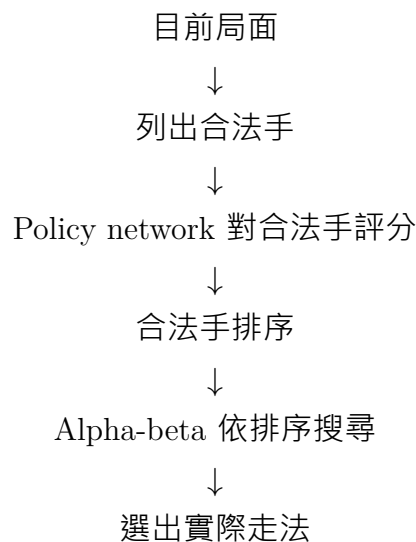
這些手工特徵提供基本棋力，使 AI 即使沒有神經網路，也能以 alpha-beta 下出合理走法。

手工評估函數屬於 rule-based / knowledge-based 設計。它和神經網路的差異在於，手工評估明確寫入人類理解的棋理，例如子力、玉安全、機動力與持駒壓力；神經網路則從資料中學習較難明確寫出的模式。shogiAI 同時使用兩者，是因為它們擅長的工作不同：policy network 擅長從大量棋譜中學「像高手會考慮哪些手」，手工評估函數則在搜尋葉節點提供穩定、可解釋、速度快的局面分數。

這種混合設計也降低了 value head 缺席的影響。既然目前模型不輸出 value，葉節點仍需要分數來源；手工評估函數提供基礎分數，quiescence search 修正戰術交換造成的 horizon effect，而 policy network 則提升搜尋排序效率。

18 Policy Network 如何結合 Alpha-beta

Policy network 不直接取代 alpha-beta，而是作為 move ordering。



Alpha-beta 的效率非常依賴走法排序。如果好手越早被搜尋， α 或 β 越快更新，就能越早剪掉其他分支。目前模型 Top-5 accuracy 達 74.489%，表示它能有效把實戰合理手排在前面。

這樣設計的核心理由是分工。神經網路擅長快速估計「這個局面下哪些手常被選擇」，但它本身不會展開變化，也容易忽略短期戰術。Alpha-beta 擅長計算變化與戰術，

但若走法順序不好，分支數會爆炸。把 policy network 放在 alpha-beta 前面，可以用資料驅動的方式改善排序，再用搜尋確認戰術正確性。

因此，policy network 在系統中不是最終裁判，而是「搜尋導覽」。它把可能有價值的路線排在前面，alpha-beta 則負責往下檢查那些路線是否真的成立。這種設計比單純使用神經網路選手更安全，也比純手工 alpha-beta 更有效率。

19 Opening Book

除了 policy network，AI 開局也可使用 opening book。Opening book 由 MySQL 統計前若干手常見局面與下一手頻率。若目前 SFEN 在 book 中，且某走法出現次數達到門檻，AI 可以直接採用該走法。

Opening book 的優點是：

- 開局階段快速，不必消耗搜尋時間。
- 走法來自真實棋譜，較符合人類常見戰型。
- 可避免早期搜尋深度不足導致怪異開局。

20 模型互打與評估

`src/compare_ai_models.py` 可讓兩個模型互相對弈。它會交替先後手，避免先手優勢影響判斷。輸出包含：

- 新模型勝場。
- 舊模型勝場。
- 和棋數。
- 新模型得分率。
- 平均手數。
- 每盤棋 USI 手順。

需要注意的是，Top-k accuracy 衡量的是「模仿實戰下一手」，模型互打才更接近棋力評估。但模型互打需要較多局數、固定深度與固定時間限制，結果才有統計意義。

21 前端展示與應用

前端 AI 對弈頁面會顯示：

- 搜尋評分。
- 搜尋深度。
- 搜尋節點數。
- PV，也就是主要變化。
- Policy 候選手與機率。

前端已移除「局面值」欄位，因為模型沒有輸出 value。這能避免使用者誤以為神經網路仍在估計勝率。

22 限制與改進方向

22.1 模型容量限制

目前模型是小型 CNN，訓練與推論速度快，但容量有限。若要提升 policy 準確率，可改用 residual CNN 或 attention-based 架構。

22.2 資料標籤限制

模型學的是人類實戰下一手，而不是數學上的最佳手。同一局面可能有多個好手，但資料只標一手。因此 Top-1 不應被解讀為唯一棋力指標。

22.3 Legal Move Mask

目前模型輸出所有 13,689 類，推論時再挑合法手。未來可在 loss 中加入合法手 mask 或改良 label 設計，讓模型更專注於實際可行走法。

22.4 自我對弈

若要讓模型不只模仿人類，可加入 self-play。透過目前 AI 搜尋產生新棋譜，再用結果訓練模型，逐步提升棋力。

22.5 Value 的未來可能

Value head 目前被移除，不代表永遠不能使用。若未來要重新嘗試 value，建議不要直接用最終勝負套到所有中間局面，而是使用搜尋評分、自我對弈結果、或更細緻的 bootstrap target。這樣 value 訊號會比單純終局勝負更穩定。

23 結論

shogiAI 的數據科學流程由資料庫、資料前處理、特徵工程、policy-only CNN、opening book 與傳統棋類搜尋組成。早期 policy/value 模型已調整為 policy-only 模型，神經網路專注於從大量實戰棋譜中學習下一手分布，並將 policy 分數用於 alpha-beta 搜尋的走法排序。截至 2026-06-23，遠端資料庫包含 80,480 局與超過 933 萬筆局面資料；目前完成訓練的資料集包含 4,855,173 筆樣本，測試 Top-1 accuracy 達 41.103%，Top-5 accuracy 達 74.489%。整體系統具備從資料蒐集、資料庫管理、模型訓練到 AI 對弈的完整流程。

A 常用指令

A.1 匯出資料集

```
python src\export_policy_dataset.py ^
--output out\policy_dataset.policy_only_latest.npz ^
--host 140.135.65.53 ^
--port 3306 ^
--user 11211213 ^
--database DB11211213
```

A.2 訓練 policy-only 模型

```
python src\train_policy.py ^
--input out\policy_dataset.policy_only_latest.npz ^
--output out\policy_model.policy_only.pt ^
--batch-size 256
```

A.3 啟動 AI 瀏覽器

```
python src\csa_browser_api.py ^
--source mysql ^
--db-host 140.135.65.53 ^
--db-port 3306 ^
--db-user 11211213 ^
--db-name DB11211213 ^
--policy-model models\policy_model.pt ^
--policy-order-ply 2
```

A.4 模型互打

```
python src\compare_ai_models.py ^  
  --old-model out\policy_model.prev.pt ^  
  --new-model models\policy_model.pt ^  
  --games 20 ^  
  --depth 3 ^  
  --time-limit-ms 1000
```